Школа анализа данных

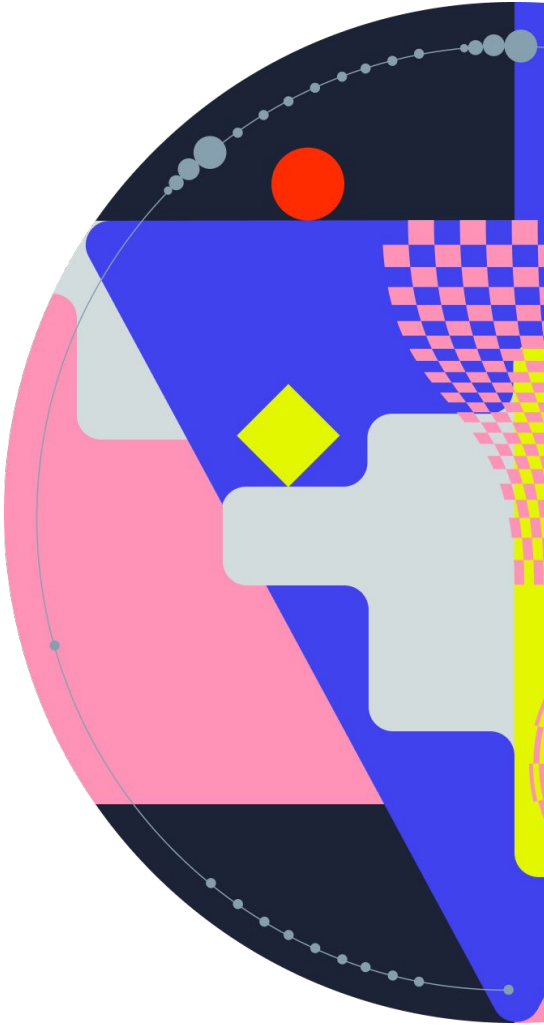
Lecture 5.2

Production Engineering for LLM Agents

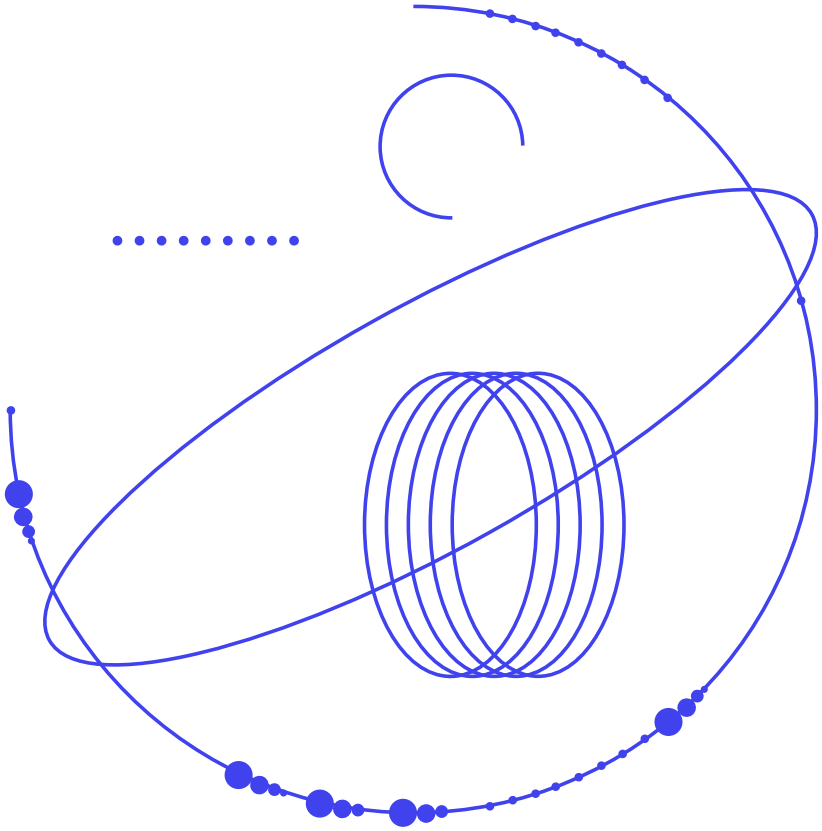
Kirill Vlasov
Product Manager in AI Studio Yandex Cloud

ACT 4

Formalisation & Decision trees

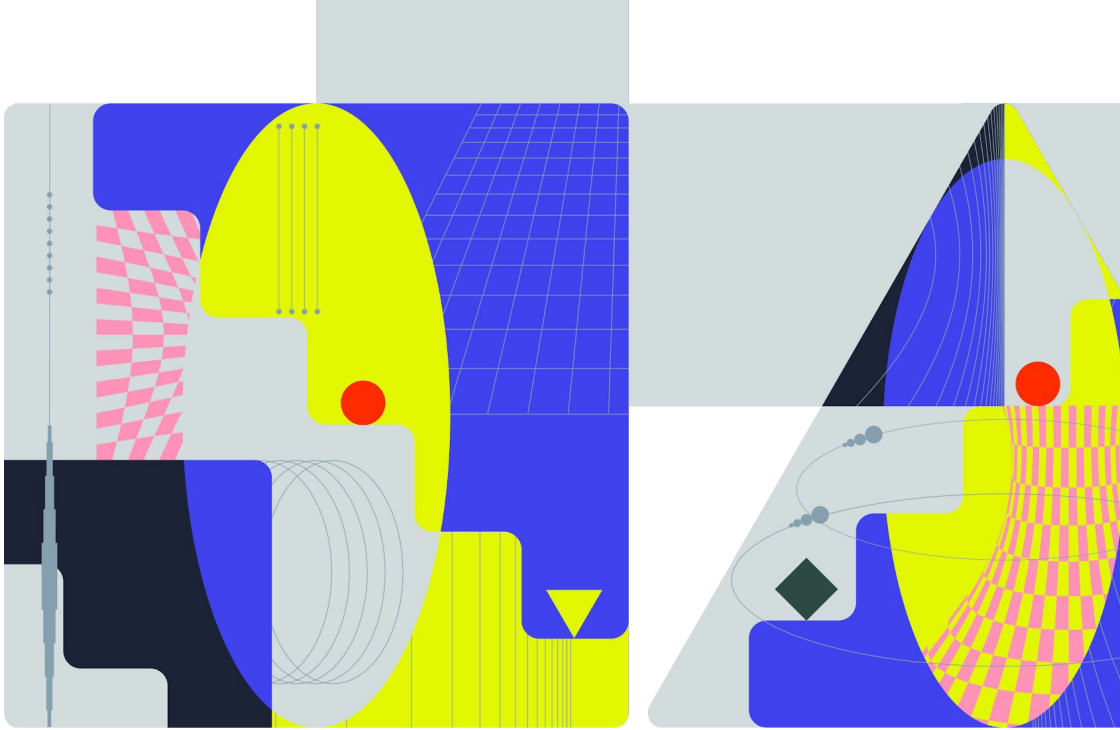


Imagine this…



An MLE and a Backend Developer walk into a bar. A PM is already waiting and says:
"We need an AI Agent…"

Do You Actually Need an Agent?



Does it need autonomy?

If NO → Pipeline / workflow

Does it need external tools?

If NO → Single LLM call

Does it need multi-step reasoning?

If NO → Prompt chain

AI Agents ≠ Software

Software Developing

- Same input → same output
- Unit tests, binary pass/fail
- Bugs are reproducible
- Ship it → it works
- Engineer programs **behavior**

AI Agents

- Same input → different output
- Statistical testing (94% success rate)
- Errors may not reproduce
- Ship → monitor → iterate
- Engineer designs **environment**

AI Agents ≠ Software

Software Developing

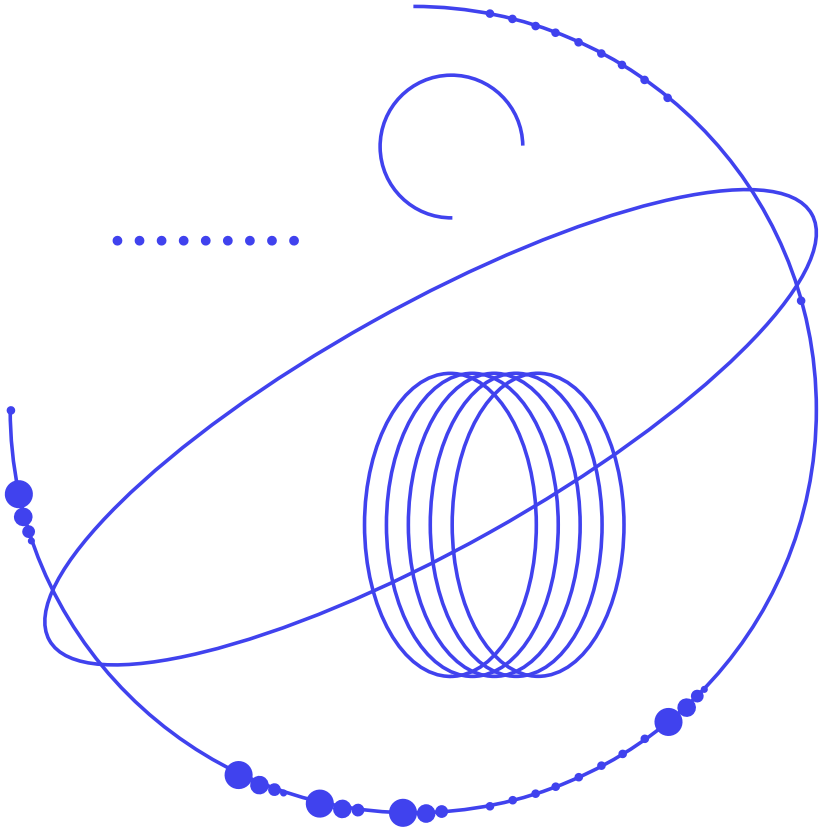
- Same input → same output
- Unit tests, binary pass/fail
- Bugs are reproducible
- Ship it → it works
- Engineer programs **behavior**

AI Agents

- Same input → different output
- Statistical testing (94% success rate)
- Errors may not reproduce
- Ship → monitor → iterate
- Engineer designs **environment**

AI Agents = ML + BackEnd

Formalize Before You Code



Before writing a single line — lock these down:

- **Input** — What comes in? (text / voice / document / structured data)
- **Output** — What comes out? (answer / action / document / API call)
- **Scope** — What the agent does and does NOT do
- **Success criteria** — How do you know it works? (metric, not "seems ok")
- **Edge cases** — What if input is garbage? Tool is down?
- **Human fallback** — When does it hand off to a person?

No formalization → no eval → no production. Just a demo

Agent Economics

Define Value first:

- What does a human do today? How much does it cost? How long does it take?
- What changes with an agent? (speed / cost / quality / scale)
- What KPI does this improve?

Then count Costs:

- **Inference:** tokens × price × steps
- **Infrastructure:** storage, RAG, compute
- **Hidden:** human verification, prompt engineering time, incident response

Cost per Session:

```
CPS = (tokens × price_per_token) × avg_steps
        + tool_call_costs
        + infra_per_session
        + human_review_cost × escalation_rate
```

Cost per Successful Resolution:

```
CPR = CPS / success_rate
```

Agent Economics

Example: Customer Support Agent

- Average: 5 steps, ~1,000 tokens per step
- GPT-4o: $2.50/1M input, $10/1M output
- 5 steps × 1,000 tokens ≈ 5,000 in + 2,500 out
- LLM cost: ~$0.04 per session
- Tool costs (API, DB): ~$0.005
- Infrastructure: ~$0.005
- Human review (10% escalation, $30/hr, 5 min): $0.25
- **Total CPS: ~$0.30**

Compare:

- Human support: $15–25/hr, handles 4–8 tickets/hr = **$2–6 per resolution**
- Agent: **$0.30** per session — 13x cheaper

BUT: only for requests the agent solves correctly.

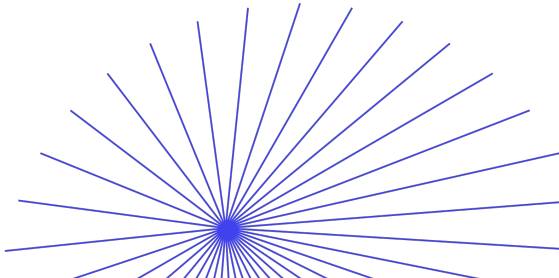
Cost per Successful Resolution:

- CPR = CPS / success_rate
- At 70% success: $0.30 / 0.70 = **$0.43**
- At 50% success: $0.30 / 0.50 = **$0.60** + rework cost

If 30% of answers are wrong and need rework — the economics change.

When an agent does NOT pay off:

- Success rate below 60% (too much rework)
- Volume under 100 tasks/month (won't recoup dev cost)
- Tasks too diverse (need 50 prompts instead of 1)
- Cost of error is high (human verification eats all savings)



Quality — Three Pillars

Prompt Engineering

Your prompt is the API contract between you and the model. Structure it: role, context, instructions, constraints, tools, output format, examples.
Version it and test every change.

Evals

Systematic measurement of whether your agent actually works. A test suite that runs automatically, scores outputs, and gates deployments.

Tracing

Without traces you don't know what happened between input and output. Log every LLM call, every tool call, every reasoning step.

Offline vs Online Eval

	Offline	Online
Inputs	Your test suite	Real traffic
Expected output	Known	Unknown
When	Before deploy (CI/CD)	After deploy (continuous)
Catches	Regressions, known failures	Drift, edge cases, real-world distribution

Online eval — how to measure without expected outputs:

- **LLM-as-judge:** sample production traces → separate model scores quality, safety, correctness
- **User signals:** thumbs up/down, repeat contacts (= failed resolution), escalation rate
- **Proxy metrics:** task completion rate, time to resolution, cost per session
- **A/B testing:** new prompt vs old on 5% traffic + shadow mode

"Agent can solve these tasks" ≠ "Agent solves real user tasks"

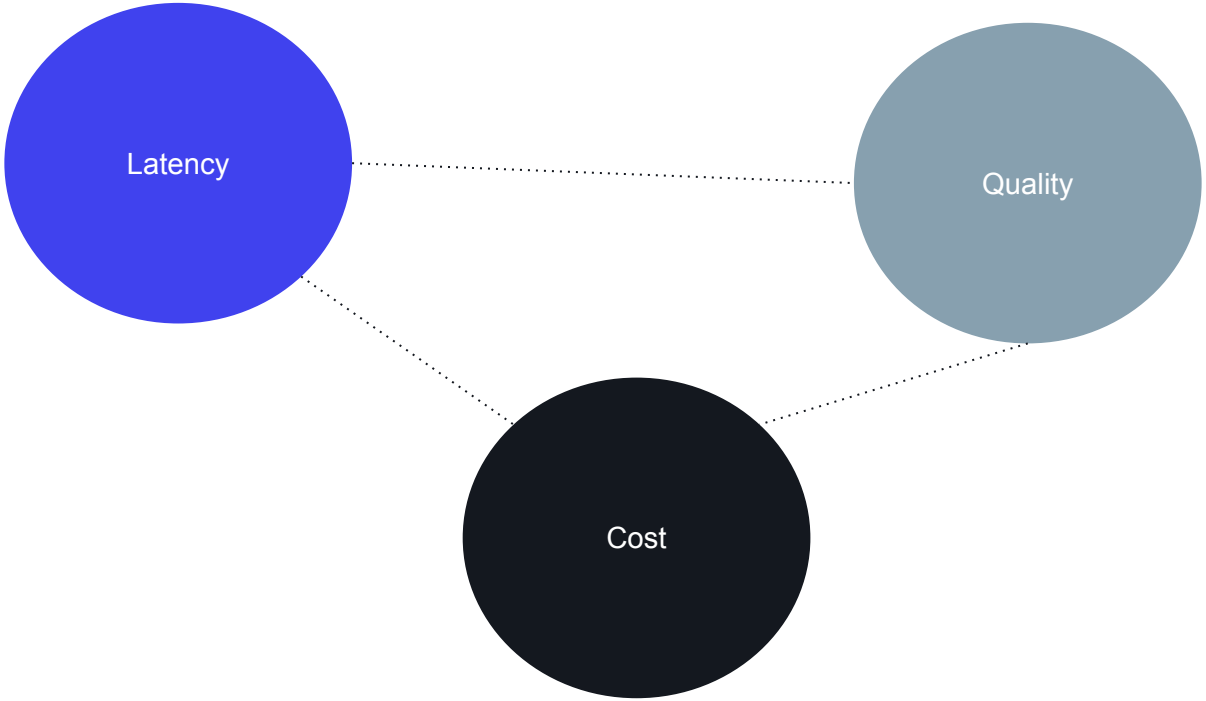
Tracing Platforms: Feature Matrix

	Langfuse	LangSmith	Arize Phoenix	Braintrust	Helicone
Focus	Tracing + prompt mgmt	Tracing + eval (LangChain)	ML monitoring + LLM	Eval-first + tracing	Gateway + cost tracking
Open source	MIT, self-host, no limits	No (SaaS, self-host Enterprise)	Yes (OpenInference)	No (SaaS)	Partially
Agent tracing	Multi-step, nested spans	Deep for LangChain/LangGraph	Session → trace → span	Nested spans, timeline replay	Basic (no chain-of-thought)
Online eval	LLM-as-judge, custom scorers	LLM-as-judge, datasets	Drift detection, alerts	Scorers in CI (GitHub Action)	No
Lock-in	Minimal	High (LangChain ecosystem)	Minimal	Medium	Minimal
Best for	Custom stack, self-host	LangChain teams	Production monitoring, drift	Eval in CI/CD, regression testing	Quick start, cost tracking

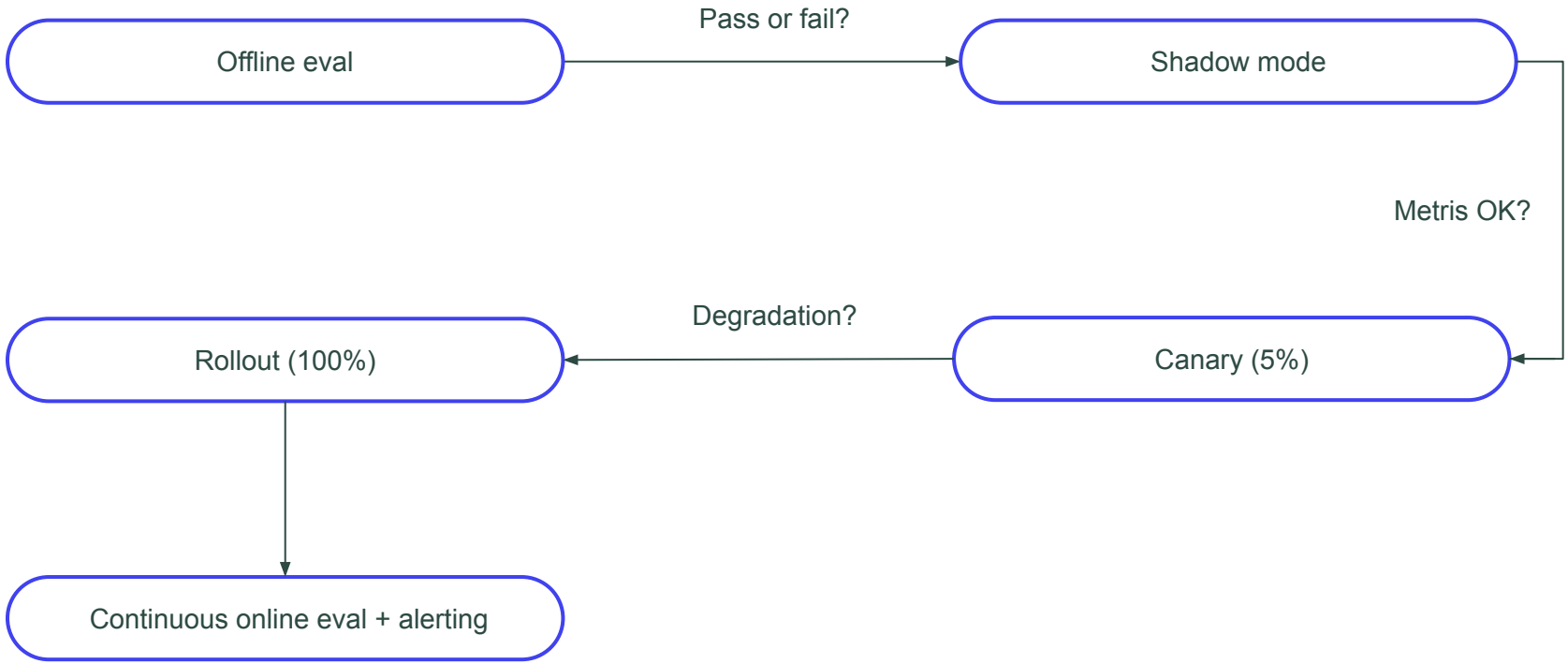
Decision tree:

- Already on LangChain? → **LangSmith**
- Need self-host / data control? → **Langfuse**
- Focus on production drift? → **Arize Phoenix**
- Eval gates in CI/CD? → **Braintrust**
- Just need cost tracking fast? → **Helicone**

Latency vs Quality vs Cost



Acceptance Pipeline



Agent Stack

Layer 5: UX & Trust: what the user sees

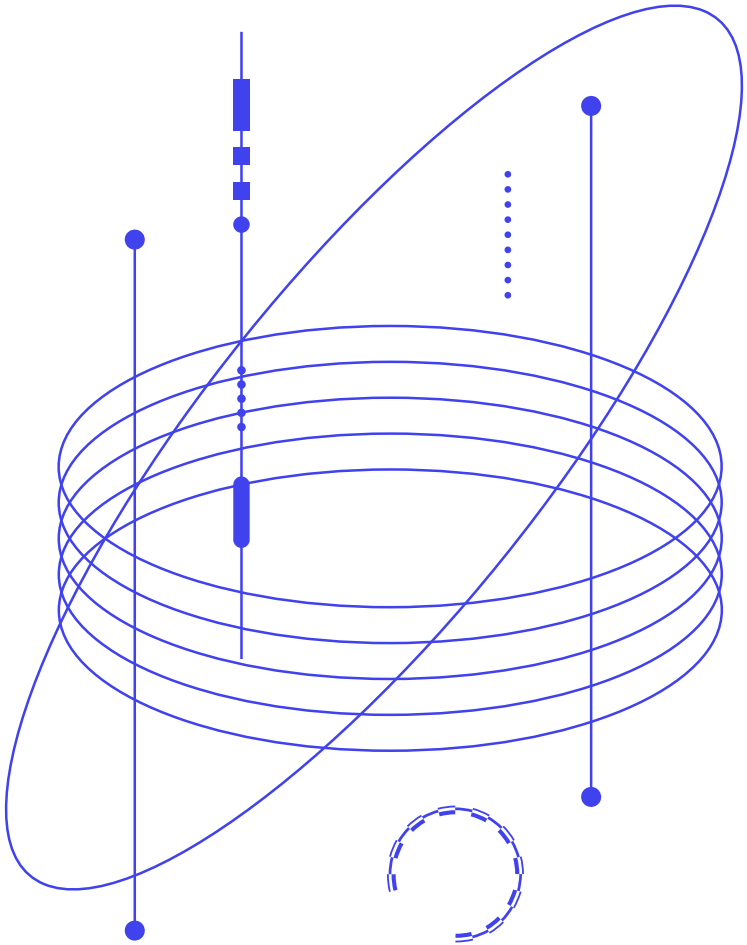
Layer 4: Orchestration: how agents and steps coordinate

Layer 3: Tools & Integrations: how agents act on the world (MCP, APIs, databases)

Layer 2: Memory & State: conversation history, RAG, long-term memory, session state

Layer 1: Model & Inference: LLM, routing, caching

Layer 0: Infrastructure: compute, networking, monitoring

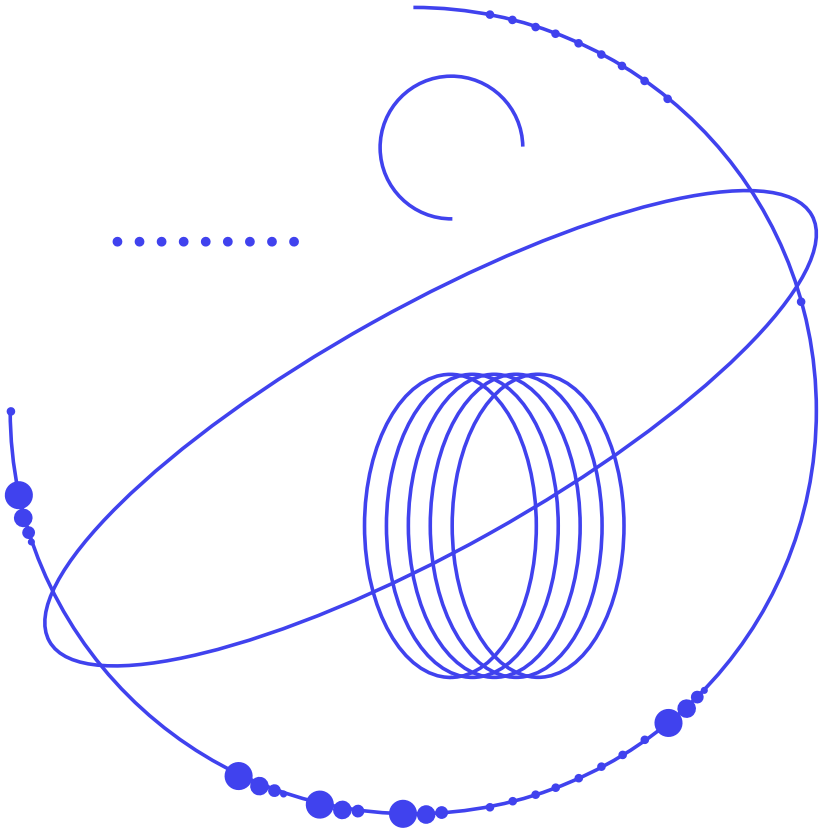


Control Plane for Agents

Kubernetes has a control plane to manage clusters.
Agents, especially multi-agent systems, need one too.

Components:

- Registry: which agents exist, their tools, permissions,
  and access scope
- Router: which agent handles which request (incl. model routing)
- Rate limiting & budgets: max tokens per session, max cost per user, circuit breakers
- Observability: traces, logs, metrics, fleet-level, not per-agent
- Governance: audit trail, approval workflows, compliance checks



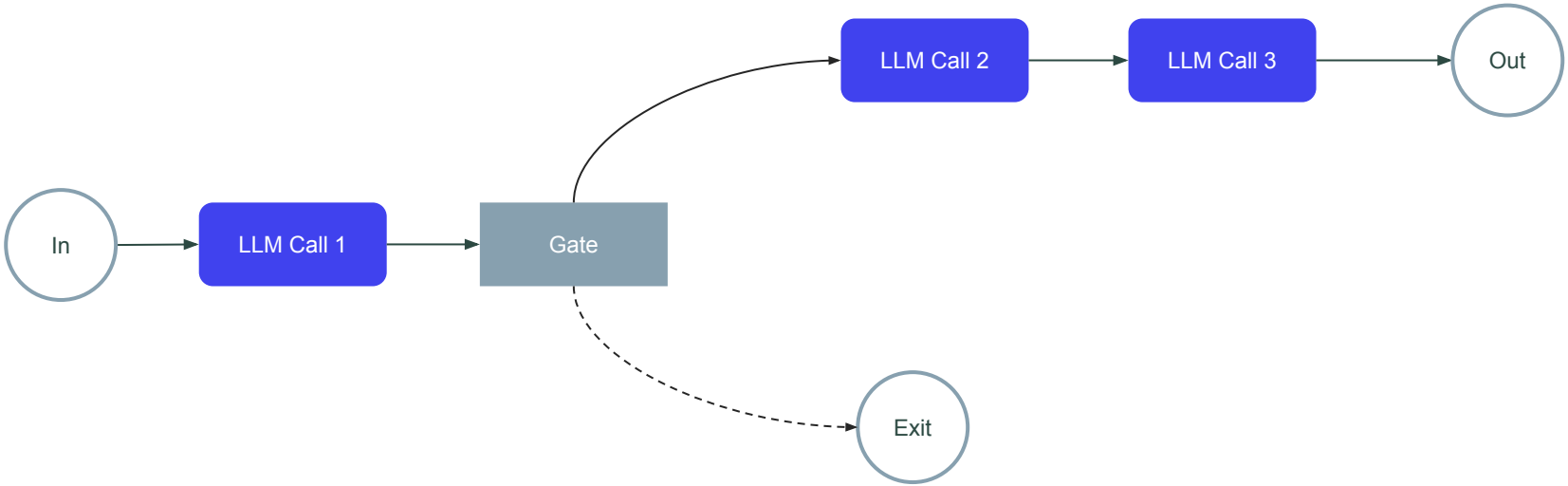
Decomposition — How to Split Into Agents

Same as software engineering: high cohesion, low coupling. But with nuances for stochastic systems.

#	Rule	Example	Anti-pattern
1	One agent = one domain	FlightAgent ≠ BaggageAgent	God Agent: one agent knows everything, 5-page prompt
2	Couple through interfaces, not state	Handoff with contract: `{flights: [...], intent: "..."}`	Agents reading each other's internals
3	One is better than two when context is shared	Search + booking tightly coupled → one agent, two tools	Two agents constantly passing context back and forth
4	Each agent testable in isolation	PolicyAgent works without FlightAgent (mock inputs)	Can only test agents together

Anthropic: Building effective agents

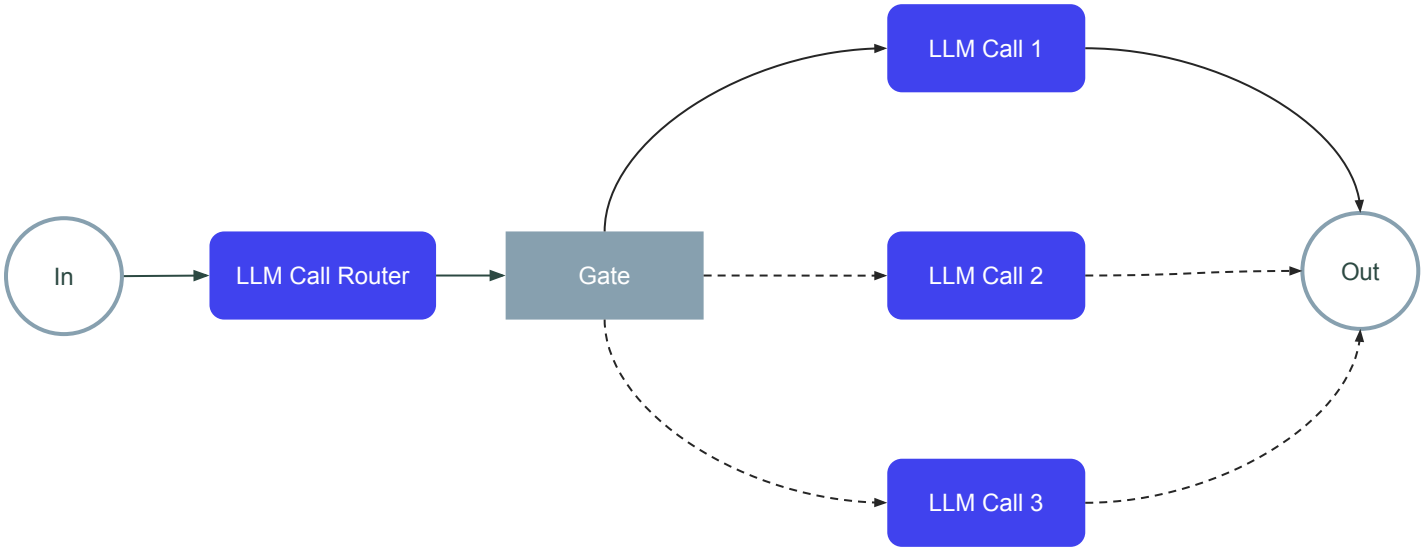
The prompt chaining workflow



Source: https://www.anthropic.com/engineering/building-effective-agents
Examples: https://github.com/anthropics/claude-cookbooks/tree/main/patterns/agents

Anthropic: Building effective agents

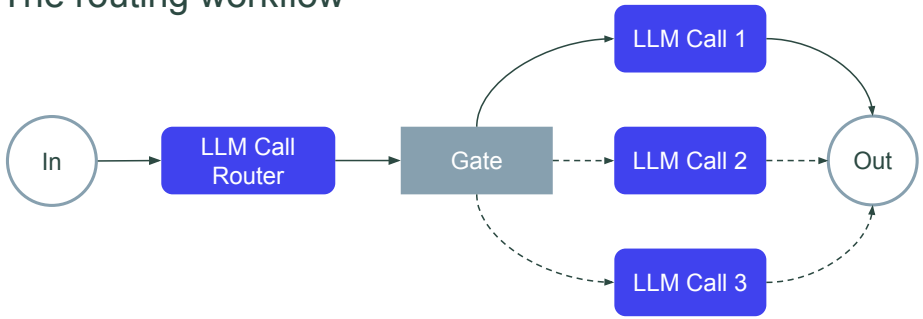
The routing workflow



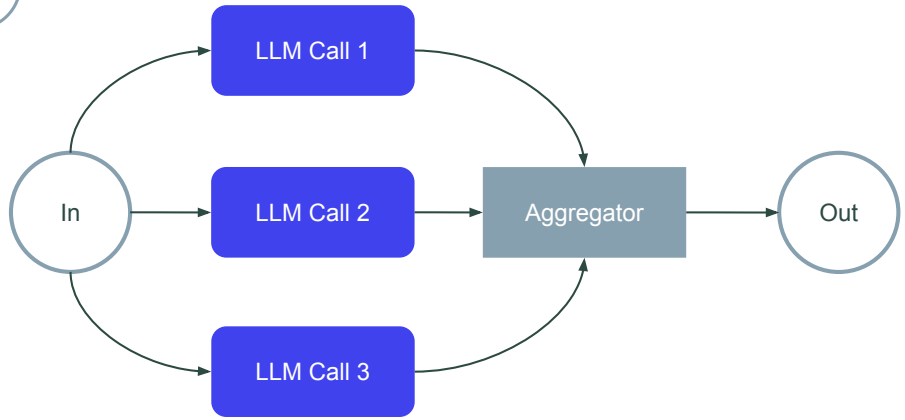
Source: https://www.anthropic.com/engineering/building-effective-agents
Examples: https://github.com/anthropics/claude-cookbooks/tree/main/patterns/agents

Anthropic: Building effective agents

The routing workflow



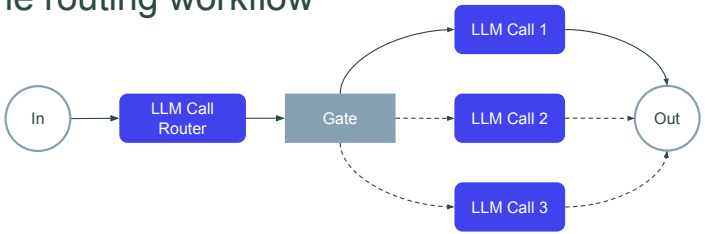
The parallelization workflow



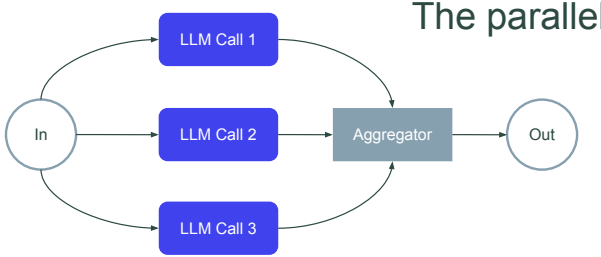
Source: https://www.anthropic.com/engineering/building-effective-agents
Examples: https://github.com/anthropics/claude-cookbooks/tree/main/patterns/agents

Anthropic: Building effective agents

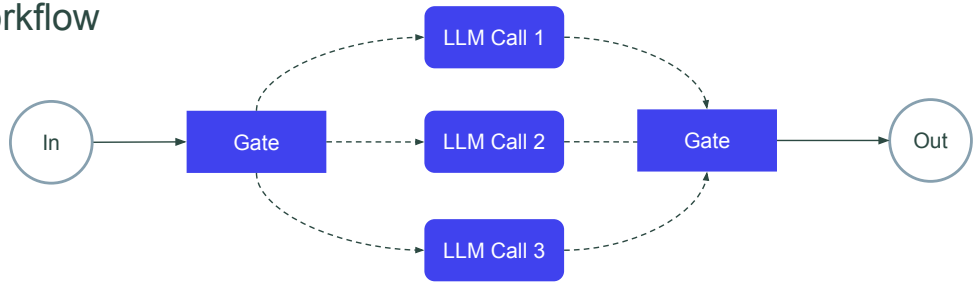
The routing workflow



The parallelization workflow



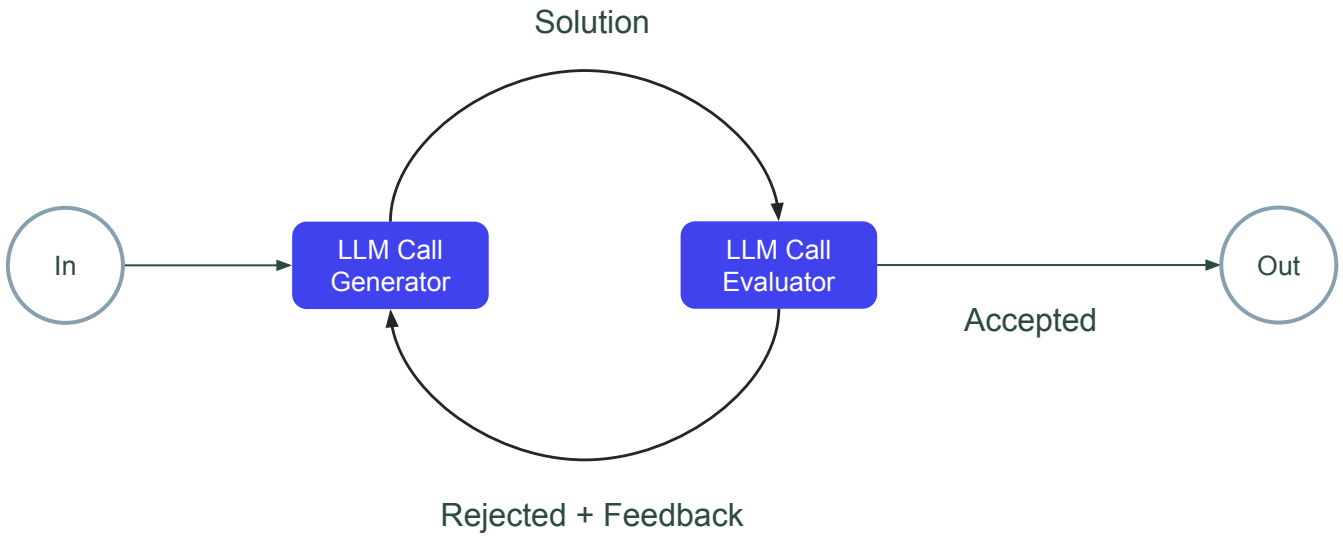
The orchestrator-workers workflow



Source: https://www.anthropic.com/engineering/building-effective-agents
Examples: https://github.com/anthropics/claude-cookbooks/tree/main/patterns/agents

Anthropic: Building effective agents

The evaluator-optimizer workflow



Source: https://www.anthropic.com/engineering/building-effective-agents
Examples: https://github.com/anthropics/claude-cookbooks/tree/main/patterns/agents

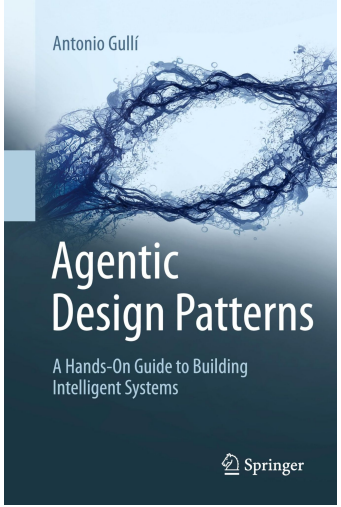
Google Patterns — Agents as Composable Units

Three levels of composition:

- **Single Agent** — one agent + N tools. 80% of tasks are solved here. Don't overcomplicate until it breaks.
- **Multi-Agent** — specialized agents with roles. Coordinator decides who handles what (LLM-driven delegation or explicit routing).
- **Agent-as-Tool** — Agent B is wrapped as a tool for Agent A. A calls `check_policy()` and gets `{allowed: true, fee: 3000}`. Doesn't know there's a full RAG + LLM pipeline inside. Encapsulation at agent level.

Two delegation mechanisms in Google ADK:

- **LLM-Driven Delegation:** model reads sub-agent descriptions, picks who to delegate to. Flexible, but model can pick wrong.
- **AgentTool (Explicit):** agent B is wrapped as a callable function. Predictable, clear contract, but rigid.



A Hands-On Guide to Building Intelligent Systems, Antonio Gulli

https://docs.cloud.google.com/architecture/choose-design-pattern-agentic-ai-system
https://developers.googleblog.com/developers-guide-to-multi-agent-patterns-in-adk/

Choosing a Framework (ADK)

Paradigms and Comparison:

GRAPH

LangGraph

Nodes → Edges.
You draw the blueprint. Max control, max boilerplate. Stateful graphs with checkpointing. Most mature (80K+ stars, 2+ years in production).

CONVERSATION

CrewAI, AutoGen

You assemble a team. Agents talk to each other. CrewAI: roles + tasks, fast prototyping. AutoGen: GroupChat, debate, consensus. Most intuitive, least predictable.

COMPUTER

Claude Agent SDK

bash + files. You provide an environment. Model decides how to act. Subagents, MCP tools, Computer Use. Minimal orchestration, maximum autonomy. Powers Claude Code.

HIERARCHY

Google ADK, OpenAI SDK

Boss → Workers.
You build a company. Coordinator delegates to specialists. ADK: workflow agents (Sequential/Parallel/Loop) + A2A protocol. OpenAI SDK: 4 primitives, fast start, OpenAI lock-in.

MCP-NATIVE

Strands (AWS)

MCP → everything.
One connector for all: tools, integrations, other agents. Deep AWS integration. Youngest framework, strongest bet on MCP as the universal standard.

Timeline

23	24	25	26
GPT-4: $60/1M tokens	GPT-4o: $10/1M Claude 3: $15/1M	GPT-4o-mini: $0.15/1M DeepSeek: $0.14/1M Inference: −78% за год	?
Value = MODEL	**Value = MODEL + PROMPT**	**Value = The SYSTEM Around the Model**	